FACULDADE DE CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE NOVA DE LISBOA

DEPARTAMENTO DE INFORMÁTICA

The BFS File System

(Mini-Project, due 7 December 2022, 23:59)

Please read this document carefully.
Your productivity (and maybe even success) strongly depends on the attention you gave it.

1. Objective

The goal of this assignment is to implement a simple file system, dubbed Basic File System (BFS), which is inspired by some UNIX-like file systems (MinixFS and UFS). BFS does not use the full features of the abovementioned FSs, like permissions' checking, dates of file creation, access, modification, etc.. It is based on the work previously carried in practical classes, extending and refining it. 😊

The next sections are organized in **four** parts.

1. The first part defines the BFS "on-disk" format.
2. The second part is the assignment itself and describes the task you must complete, dealing with operations (details later) on regular files and directories (successful completion awards you a 200 points or 20,0 valores).

2. The BFS file system

The BFS (Basic FS) file system is stored in a *disk* simulated by a *file*. It is possible to read and write *disk blocks* to this simulated disk, corresponding to *fixed size data chunks*.

A disk formatted with the *BFS format* has the following organization (see Section 2.2 for further details):

- A *super-block* describing the disk's organization
- A bytemap to manage inodes
- The next ***I*** blocks contain the i-node table, where the used i-nodes contain metadata on existing files in the FS. This table occupies a fixed number of disk data blocks, specified in an argument to the format command
- The first disk block after the i-node area is the start of a bytemap that controls the allocation of data clusters
- Finally, the remaining disk blocks are used for the files' contents

Figure 1 shows an overview of a disk with a total of **40** *blocks*, after being initialized with the *format* command.

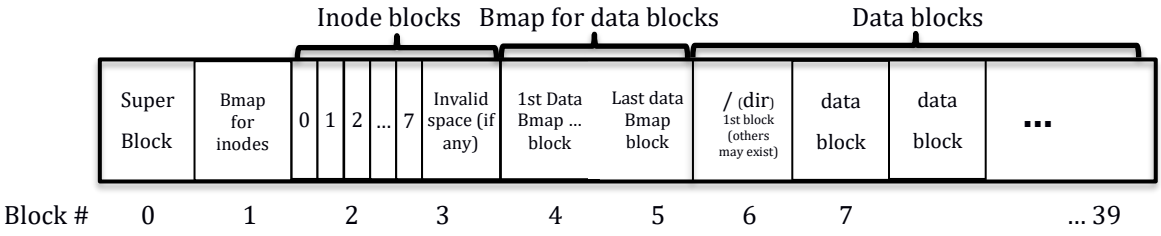

Figure 1

2.1. BFS layers

Figure 2 depicts the BFS abstract layers; not all are available to you (it would be too complex for the time we have for this assignment) but, of those which are available, each one is implemented with a set of "independent" C files, usually just two: a file ending in `.c` and another in `.h`.

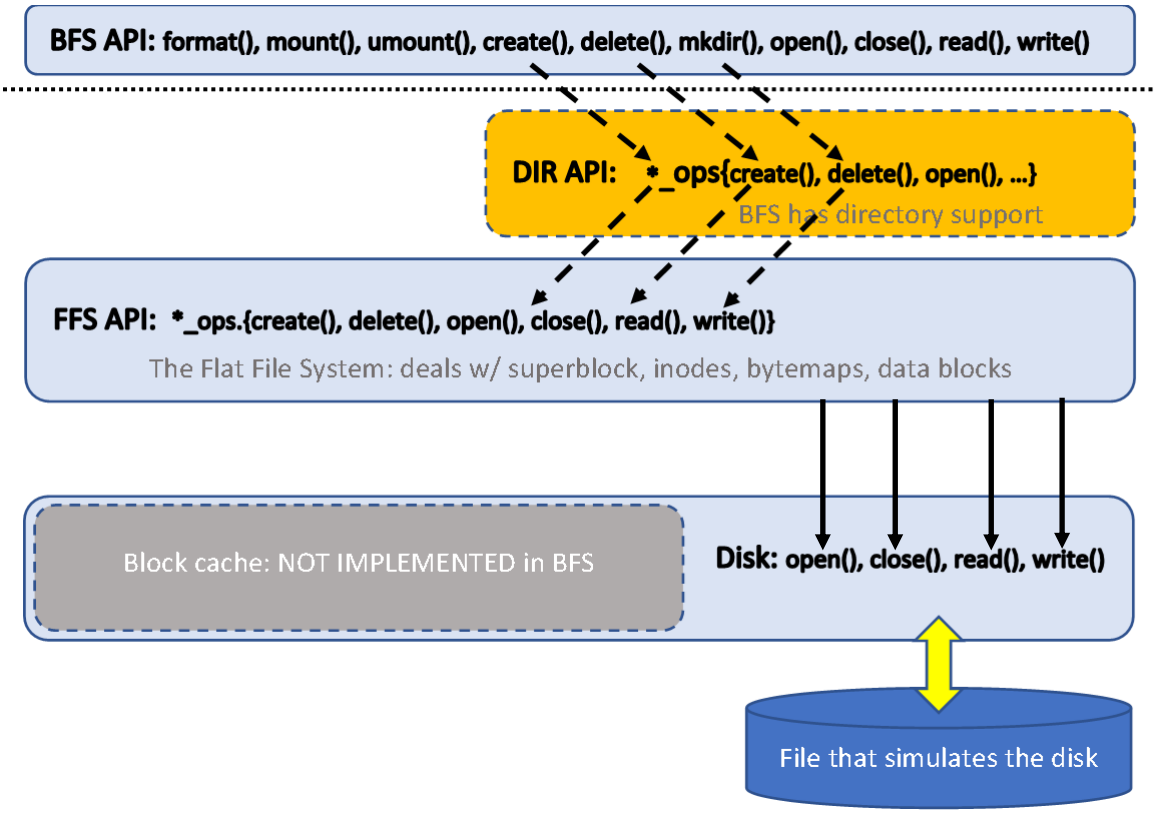

Figure 2 (Note: **some API functions may not be implemented or have different names**)

For the purpose at hand, the supplied files are, at the disk (device) level, `disk_driver.{c,h}`, at the file system level, `ffs_super.{c,h}`, `ffs_bytemap.{c,h}`, `ffs_inode.{c,h}`, and `ffs_file.{c,h}` and, at the BFS level, `bfs_dir.{c,h}` and the "topmost level" `bfs_lib.{c,h}`.

Notice that some functions are available in the user-level API, i.e., above the dotted line (which fakes a separation between user and kernel layers), while others are not (e.g., those available at the Disk layer). Also, notice that directories are supported, so some functions, e.g., `create()` indirectly call functions in other modules (e.g., DIR) before landing on the most important module, the FFS. You will deploy your code using an object-oriented programming style in C, and have functions that look like they have the same name in various "places".

VERY IMPORTANT NOTE: The names of structures, functions, fields and parameters, defined hereafter may, or may not, be exact. You **must** always look at the include files `.h`, for the "absolute truth" 😊.

2.2. The on-disk BFS layout

The *BFS on-disk organization* is described (see also Figure 1 and `ffs_super.{c,h}`) in the following points:

- Block 0 is the *super-block* and describes the disk organization. All values in the superblock are *unsigned integers* (4 bytes) and they are placed in the super-block *in the following order* (see the `struct superblock` later in the document). The "base" information is:
    - *fsmagic* "magic number" (`0xf0f03410`) - used to verify if the file image is a BFS formatted disk
    - *nblocks* - total number of blocks (defines the disk size, in blocks)
    - *nbmapblocksinodes* - number of blocks reserved to store the bytemap for the inodes (always 1 in BFS)
    - *ninodeblocks* - number of blocks reserved to store i-nodes
    - *ninodes* - total number of i-nodes
    - *nbmapblocksdata* - number of blocks reserved to store the bytemap for data blocks (computed)
    - *nclusters* - number of "user-available" clusters (each with *clusterSize* contiguous data blocks)
    - *clusterSize* – number of blocks in a cluster (1, 2, 4, 8, … 2^n)

However, some more pieces of information have been added, to simplify the access to the different structures in the disk. These are "redundant" in the sense that they can be computed from the (above) previous data items, but having it already computed is a bonus.

- *startInArea* – disk block (absolute address) where the inode area starts – i.e., i-node table offset
- *startDtBmap* – disk block (absolute address) where the data bytemap area starts
- *startDtArea* – disk block (absolute address) where the data area starts
- *mounted* – indicates if the disk is mounted, or was left mounted (due to, e.g., power failure or bug)

- The block at 1 is used to store the bytemap that manages i-nodes (only 1 is used)
- The next *ninodeblocks,* starting at (and including) block 2, contain i-nodes. Each **i-node** in the **i-node table** has the following contents (full description down, in 2.3.1, and in `ffs_inode.h`):
  - contains metadata about the i-node (in use or free, plus other information)
  - *size* – indicates the length of the **file** in bytes
  - an area for data pointers that contains information about the block numbers of the data blocks occupied by the file.
- Following the i-nodes, a certain number of blocks is used to store the bytemap that manages data block clusters (computed, see the details on the **`format()`** operation further down)
- The next **`nclusters`** are used for storing the files' contents (including directories).

The *BFS superblock* and the required operations are described below (see `ffs_super.{c,h}`):

```
struct super {
  unsigned int fsmagic;
  unsigned int nblocks;          // get it from disk driver
  unsigned int startInBmap;      // Always 1 in BFS
  unsigned int sizeInBmap;       // Always 1 in BFS
  unsigned int startInArea;
  unsigned int sizeInArea;       // User format parameter (in blocks)
  unsigned int ninodes;
  unsigned int startDtBmap;
  unsigned int sizeDtBmap;
  unsigned int clusterSize;      // User format parameter (in blocks)
  unsigned int startDtArea;
  unsigned int nclusters;
  unsigned int mounted;
};
```

To read/write the *superblock* from/to the disk, a *union* is used:

```
union sb_block {
  struct super sb;
  unsigned char data[DISK_BLOCK_SIZE];
};

/* operations on superblock structures
  create: To be called by format(), thus requires that the SB is NOT mounted as it
          overwrites in the in-memory SB structure.
    parameters:
     @in: disk size (nblocks), number of blocks to allocate ... etc.
     @out: pointer to superblock structure

  read: read: reads in a SB from disk, overwrites the in-mem SB structure.
        Requires mounted
    parameters:
     @out: pointer to superblock structure
  ...         ...           ...
*/

struct super_operations {
  int (*create)(struct super *sb);
  int (*read)(struct super *sb);
  ...         ...           ...
};
```

2.3. Other BFS structures: bytemaps, i-nodes and directories

The other structures are: at the FFS, i-nodes and bytemaps (two of them); at DIR, directories.

2.3.1 i-nodes

i-nodes are stored in *ninodeblocks* blocks, each block holding some number of i-node structures (see below), and care has been taken so that all i-node entries are significant – the total number of i-nodes is *ninodes*. BFS supports indexed allocation with 6 data pointers (5 direct and 1 indirect) that point to data clusters.

```
#define DPOINTERS_PER_INODE 5
struct inode {
  unsigned char  type;
  unsigned char  nlinks;
  unsigned short size;
  unsigned short dptr[DPOINTERS_PER_INODE];
  unsigned short iptr;
};
```

The purpose of **`type`** is as follows: **`0`** denotes a free inode, **`1`** a regular file, and **`2`** a directory. Directories are stored as regular files, but they can only be accessed with the appropriate functions; as an example, an **`opendir()`** of an existing directory should succeed, but an **`open()`** should fail with **`EISDIR`**; the reverse is also true, i.e., an **`opendir()`** of a regular file should fail with **`EISDIR`** (NOTE: the test programs will not be testing this; you may skip that test… or not) The purpose of **`size`** is obvious; **`dptr`** are the i-node's direct pointers, and **`iptr`** is the sole indirect pointer and **`nlinks`** is to count the number of references (links) made to the file.

And, finally, to be able to read/write blocks of inodes from/to the disk,

```
union inode_block {
  struct inode ino[INODES_PER_BLK];
  unsigned char data[DISK_BLOCK_SIZE];
};
```

As examples, an indexed file is described by a 16-byte i-node structure. So, 1 disk block packs a maximum of, exactly 😊, 32 i-nodes of the "indexed file type".

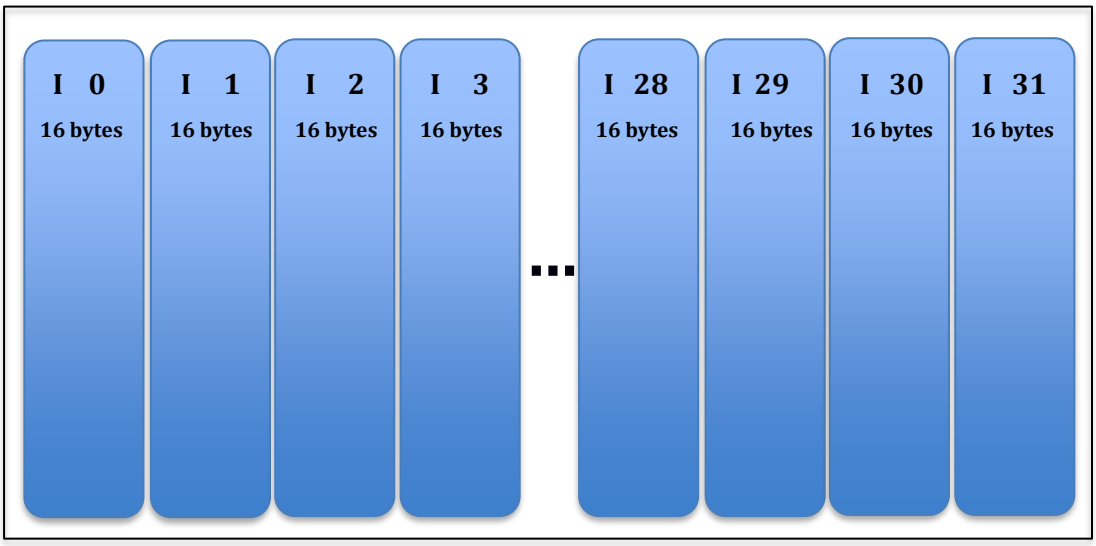

Figure 3: 32 large i-nodes adequate for indexed files, packed in a block

Notice that we have numbered the i-nodes sequentially, starting from zero. With such a scheme, it seems pretty easy to deduce which disk block to access for a particular i-node number (you do the math 😊). ***TODO***: you will have to implement all functions in **`ffs_inode.c`**.

2.3.2 Bytemaps

As you know bytemaps are arrays (or blocks, if you prefer) of bytes; in BFS, a bytemap is stored in disk blocks, but not all entries are significant – only the first *ninodes* entries, for the bytemap that is used to manage i-nodes,

and only the first *nclusters* entries, for the bytemap that is used to manage data blocks, are valid. Remember that data blocks are assigned in clusters (of **`clusterSize`** contiguous blocks), so if **`clusterSize`** is, e.g., **`4`**, to grow a file, a cluster of **`4`** contiguous "free" blocks must be found… Notice that at "the end" of the disk, a number of blocks may be left unused, as they may not be enough to form a cluster.

Why are bytemaps, and not bitmaps, that were always referred in lectures, used? To ease you programming efforts. 😊 ***TODO***: you will have tom implement all functions in **`ffs_bytemap.c`**.

2.3.3 Directories

The BFS *dentry* (abrev. of directory entry) structure is depicted below; note that when a new directory is created, two entries, **`.`** and **`..`** must be filled with the appropriate i-node pointers. i-node **`0`** is reserved for the disk's root directory and, naturally, the **`.`** and **`..`** entries must, in this particular case, point to **`0`**.

```
#define FILENAME_SIZE 14
struct dentry {
  name[FILENAME_SIZE];
  unsigned short inodeNumber;
};

/* operations on directory structures
  createF: creates a new file in the current directory
    parameters:
     @in: file name;
     @out: inode number of the new directory

  rewinddir: resets the file pointer used by readdir when scanning a directory.
    parameters:
     @out: full pathname of the current directory

  readdir: reads the next directory entry
           can be used to look for free or valid entries...
  ...        ...          ...
*/

struct dir_operations {
  int (*create)(const char *name);
  int (*opendir)(const char *name);
  int (*readdir)(const unsigned int fd, struct dentry *dp);
  ...        ...          ...
};
```

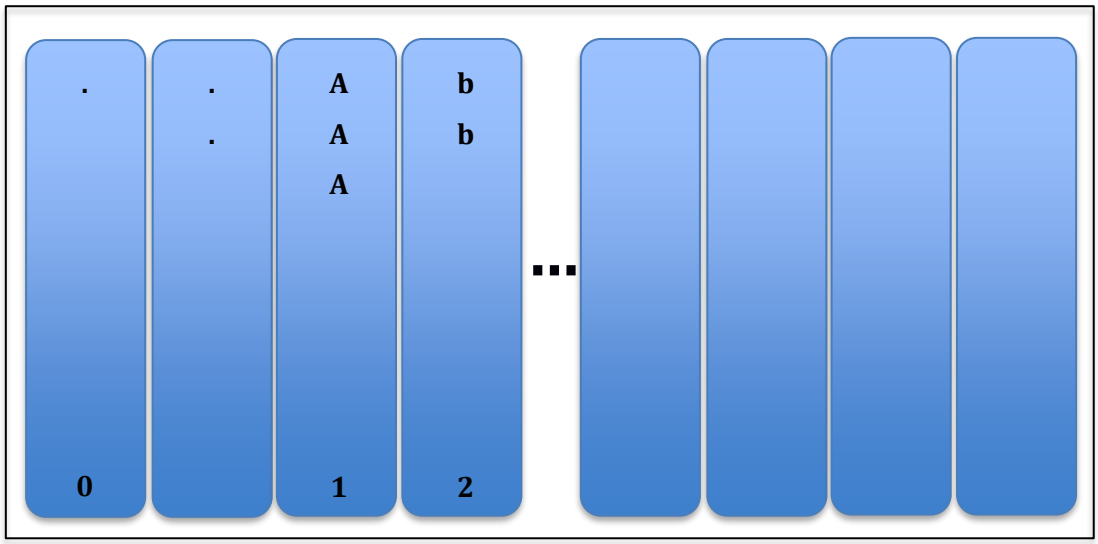

Figure 4: 32 *dentries* in the / (root) directory, packed in a cluster of size 1 (1 block). Here, two names have been created: **`AAA`** for a file with i-node 1, and **`bb`** for a file with i-node 2; also, some free entries

TODO: you will have to implement the **`readdir()`** function (in **`bfs_dir.c`**)

3. Where to start? (the speedy way)

3.1 Format a disk!

The first thing you must do is create the **`format()`** function, which will be called by the supplied main program, `testBFS.c` and starts by fill-in the information in **`struct super`** and write it to block 0. The **`format()`** function has two parameters: the **number of i-node blocks** and the **`clusterSize`**. It starts by i) calling **`disk_driver`**[1] to get the size of the disk; ii) then, given the number of blocks to use for the i-node table, it computes the number of blocks to use for the data bytemap; finally, computes all the remainder information to fill in **`struct super`**, and writes it to disk block **`0`**. That's it 😊. But, for that, you must write the **`struct super_operations write()`**, then **`read()`**, and **`debug()`**, etc..

When all these functions are implemented, you can use `testBFS` to try them and see if they produce the required outputs…

DO NOT PROCEED to implement other operations if **you have not finished & <u>tested</u>** the "format disk".

But is the **`format()`** function complete? No…

A formated disk must have a directory (and that requires an i-node) and the directory must have two entries, "**`.`**" and "**`..`**"; in BFS, the root directory must use i-node number 0 and, naturally, the 1st available free cluster, i.e., logical (data) cluster number 0. [**Uuuupssss?!** This requires directories to be implemented! Let's forget it for the time being…]

In your first implementation of the **`format()`** function, you can "directly" write i-node 0, the bytemaps, and the entries "**`.`**" and "**`..`**", but when the appropriate generic functions for those structures are completely tested, you **<u>must</u>** use them in the **`format()`** function.

What should be the size of a directory? Should it be the number of stored entries? No, because that would require all valid entries to be packed contiguously (think about that for a minute 😊). So, for a directory, size is the maximum byte length of the "reserved space", i.e., **`512*<number of clusters used>*clusterSize.`** Therefore, in Figure 4, the i-node 0 (which describes the root directory, **`/`**) must have it's size field set to 512 (supposing that the size of the cluster is 1 and the current length of the directory is 1 cluster).

How do we flag the free dentries (on the right side of Figure 4)? Well, having their **`name`** fields set to NUL (a.k.a **`'\0'`**, or zero) seems a nice ideia 😊.

3.2 Implement the `mount()` and `umount()` operations…

The **`mount()`** operation is quite simple:

a) read in the superblock to a global variable named **`super`**;

b) check the **`mounted`** field; if not set to **`0`**, report an error; else set **`mounted`** to **`1`**, and write back the superblock;

c) The END.

Naturally, the **`umount()`** operation is also quite simple; the details are for you to "guess".

A **`debug()`** operation is also quite useful, and in our case mandatory; it should print out the superblock according to the specified parameter: if **`MOUNTED`**, it should print the contents of the in-core **`super`** global variable; else, if **`UNMOUNTED`**, it should print the contents of the on-disk superblock.

[1] The documentation is included in **`disk_driver.h`**

4. Where to continue? (now in a structured way)

You may recall that in Figure 2 we have modularised our FS into 3 layers: FFS, the Flat File System layer, DIR, the directory layer, and BFS, the "user-level" API library that exposes the BFS functionalities to programmers. The objectives of the DIR and BFS modules are easy to understand; but what, exactly, is the FFS?

The FFS deals with all FS operations except directories. In the realm of the FFS there is a superblock, two bytemaps, i-nodes, and also files! The "thing" is that FFS files have all the usual file characteristics, but no names; and that's because names are in the realm of directories. So, how is a file identified in FFS? By its i-node.

And this is why you will find "program" files (with **`.h`** and **`.c`** extensions) named **`ffs_super`**, **`ffs_bytemap`**, **`ffs_inode`** and, the "topmost" one, **`ffs_file`**; altogether, they constitute the FFS layer. So, you must implement those (sub)modules first.

By now, in practical Labs, you have already seen what operations are available in the **`disk_driver`** (NOTE: the disk driver for this Mini-Projecto is slightly different, it allows I/O sizes that are multiple of the disk block's size – that is needed to operate on clusters). And you have seen i-nodes and bytemaps (and implemented some of their operations).

Now, in the Mini-Projecto, you must carefully review the **`ffs_bytemap`**, **`ffs_inode`** and **`ffs_super`** files and implement all the operations (and test them carefully, with the test program **`tstBFS.c`**).

4.1 Implement the basic "FFS" operations

Superblock. Start with the implementation of the **superblock** (SB) **operations**; test them all, using different values for the size of the **disk**, size of a **cluster**, and size of the **i-node area**: from these parameters you can compute all the information to fill the in-memory superblock structure; then, write it to disk. Test the **`mount()`** and **`umount()`** operations; print the superblock and see if all values are correct.

Bytemaps. Then, move to the **bytemap operations** – remember, there are two bytemaps, in different disk locations: the bytemap that is used to manage i-nodes is easy: it occupies 1 block, and is located at the "physical" disk block 1. However, that information is stored in the SB, so a good program does not use the constant value "1", but the information it retrieves from the SB: **`startInArea`** and **`sizeInArea`**. See that we propose that you use an array of **`struct bmapMData`** of to handle the two bytemaps – you are free not to use that proposal, as long as the two operations, **`getfree()`** and **`set()`**, work correctly. If you use the abovementioned array of **`struct bmapMData`**, you should also implement the bytemap operations **`mount()`** and **`umount()`** to initialize and clear that same structure. Test the **`getfree()`** and **`set()`** operations; print the bytemaps and see if all values are correct.

i-nodes. Now, move to the **i-node operations** and notice that now we have two i-node structures: one that is used to store/retrieve to/from disk, called **`struct inode`**, and another to hold i-nodes in memory, called **`struct IMinode`**. Test the i-node operations; print the i-nodes and see if all values are correct. Store them on-disk, unmount the disk, mount it again, and retrieve the information to check that it is still valid.

A note about **`format()`**: now that you know "everything" about clusters, bytemaps and i-nodes, remember that when you format a previously used disk, you should wipe-out every "zone" other than the superblock, i.e., you must also "clean" the bytemaps, i-nodes, and the data clusters!

4.2 Implement the other "Flat File" operations

Now that the basic FS entities (**superblock**, **bytemap**, and **i-node)** are ready (and tested!), we can proceed to the implementation of the **file** entity. Flat files may be created (thus requiring a free i-node – which requires that the i-nodes bytemap be searched for a free entry) with a type, **regular** or **directory**. Creation of a "file" must be stored, so the free bytemap entry must be set to in-use and saved to disk, together with the i-node information.

A file may be read, and/or written. Writing a file may require blocks (that is, clusters) to be allocated, which requires that the data bytemap be searched for a free cluster entry, the i-node pointer set to that entry's number (and that's where things may get complicated if you decide to handle the indirect pointer and a one-level index cluster). For now, assume you have a few functions gathered together under **`ffs_file_operations`** below,

```
struct ffs_file_operations {
  ...
  int (*format)(const char *diskname, const unsigned int nmrInoBlks, \
                const unsigned int clusterSize);
  int (*mount)(const char *diskname, struct IMsuper *imSB, const int debug);
  int (*umount)(struct IMsuper *imSB);
  int (*open)(const unsigned int inoNbr);
  int (*close)(const int fd);
  int (*write)(const int fd, const void *buffer, const unsigned int length);
  int (*read)(const int fd, const void *buffer, const unsigned int length);
  int (*create)();
  int (*delete)(const unsigned int inoNbr);
  ...
};
```

TODO: you will have to implement the following functions in **`ffs_file.c`**: **`ffs_file_unlink`** and **`ffs_file_write`**

4.2.1 Mount and Unmount

In the "**flat filesystem**" (FFS) realm our implementation of "mount" is a simple composition of two operations: a **superblock `mount()`** and (if you followed our suggestion on bytemaps) a **bytemap `mount()`**. After executing these two operations, and setting up the FFS file module own structures, everything but the directory operations can be implemented.

The "unmount" operation is also a simple call to a **bytemap `umount()`** followed by a **superblock `umount()`** and an operation to clear the FFS file module own structures.

4.2.2 Create and Delete

In the FFS layer our implementation of "create" is a simple composition of two operations: a **bytemap** operation, where we look for a free i-node and mark it in-use (and save that to disk), and an **i-node** operation, where we fill it with relevant information, and save (write) it to the correct disk block.

The "delete" is more complex: it may just be a "unlink", that decreases the **`nlinks`** field of the i-node or, if **`nlinks`** reaches 0, a more complex operation that requires i) data blocks to be freed (marking the bytemap entries as free) and ii) the i-node itself to be cleared – and both structures saved back to disk. Notice that you may now "clean" the discarded datablocks (useful for debugging) otherwise you must clean them prior to allocation (why? 😊).

4.2.3 Open and Close

You may remember that in the FSO lectures we've talked about the concept of "open files Table"; that's what we use here, in FFS – the table is an array of type "file structure" depicted below:

```
struct ffs_file {
  unsigned int valid;
  unsigned int fpos;
  struct IMinode IMino;
}

struct ffs_file openFT[MAX_OPEN_FILES]
```

That is, you have an indicator when an entry in the table is valid, the in-memory i-node for that file, and the file pointer. So, when you open a file, you locate the first free entry in the table, then use the i-node number argument to locate the i-node on disk, load it into the in-memory **`IMino`** structure, and finally set the entry's valid flag and set the file pointer to the beginning of the file (zero). The close operation is "the reverse" (think about it).

4.2.4 Read and Write

Now we can describe what happens in a read: suppose it's the first read on a file (and that file has contents, i.e., its size is greater than zero); and, suppose that you are reading a full cluster, i.e., your read is similar to

```
ercode= read(fd, buf, clusterSize)
```

Your implementation must: i) look at the table entry indexed by **fd** and get the value of the i-node's first direct pointer (that's a cluster address); ii) read into a private buffer the contents of the disk cluster at the given cluster address; iii) copy **clusterSize** bytes to **buf** (the "user buffer"); update the file pointer (by **clusterSize**); iv) and return **clusterSize**. Notice that, as described in the lectures (and the OSTEP book), the read does not access any bytemaps 😊.

Now, the write is "just the opposite of the read 😊 but, remember, you may need to allocate data clusters to store the user's contents in the buffer – therefore, you will sometimes need to access the data "clusters" bytemap: get a free cluster, set it to in-use, and write the buffer to disk on that address; you will also need to store the address in the i-node's appropriate pointer and (depending on your flush policy) immediately write the changes to disk or mark the i-node as dirty and only update it when the file is closed.

NOTE: for this assignment assume that the length of all read and write operations is **clusterSize**; therefore, only files whose size is a multiple of **clusterSize** will produce correct results.

4.2.5 `format()` finally "fully implemented"

Now that you have at least a file write operation that works with a buffer with the same size of a cluster, you can "hack" a root directory in the format operation, just do:

a) Create a union between a buffer (with the same size of a cluster) and an array of dentries
b) On dentry[0] set name="." and inode= 0;
c) On dentry[1] set name=".." and inode= 0;
d) Write that buffer to the 1st data cluster, mark that cluster as "in-use" in the data bytemap
e) Set the 1st i-node (i-node number 0) with the appropriate values (nlinks= 1, size=clusterSize, 1st data pointer with the address of the 1st cluster – i.e., zero 😊). Write the i-node. Mark that i-node as "in-use" in the i-nodes bytemap
f) The END.

4.3 Implement the "DIR" operations

We've seen that the dentry is a simple structure (see 2.3.3) and it fits nicely in a block (and therefore, in a cluster). The supported operations are:

```
struct dir_operations {
  int (*mount)(struct IMdirectory *cwd);
  int (*umount)(struct IMdirectory *cwd);
  int (*opendirI)(struct IMdirectory *cwd, unsigned int inoNbr);
  int (*openF)(struct IMdirectory *cwd, char *name);
  int (*closeF)(int fd);
  int (*readdir)(struct IMdirectory *cwd, struct dentry *dirent);
  void (*rewinddir)(struct IMdirectory *cwd);
  int (*create)(struct IMdirectory *cwd, char *name, \
                unsigned int type);
  int (*unlink)(struct IMdirectory *cwd, char *name);
  int (*mkEmptydir)(unsigned int root, unsigned int pIno);
};
```

4.3.1 Mount and Unmount

The directory "mount" operation resorts to the **ffs_open()** to open the i-node 0 and populate the root directory structure with the filename (/), i-node number and i-node, etc.

The directory "unmount" operation clears the root directory structure and calls the **ffs_close()** to perform the remaining chores.

4.3.2 Gluing all together under the BFS library, available to the user programs

To illustrate how things are glued together in BFS (that is, in the **bfs_lib.c** library), that is used by user's programs and some of our test programs, lets see some examples in the following sections. Suppose that i) you want to create a file named "F", ii) open it for writing, and iii) after some write(s), close it.

Inside BFS our abstraction of "create" is both a **directory** operation, where we store the name/inode pair, and a "**flat filesystem**" operation, where we look for an available i-node and fill it with relevant information. For now, assume you have a few functions "extracted" from other modules and gathered together under

```
int bfs_create(char *name);
int bfs_unlink(char *name);
int bfs_mkdir(char *name);
int bfs_open(char *name);
int bfs_close(int fd);
int bfs_read(unsigned int fd, void *buf, unsigned int length);
int bfs_write(unsigned int fd, void *buf, unsigned int length);
int bfs_lseek(const unsigned int fd, const int offset, const unsigned int whence);
void bfs_format(char *diskname, unsigned int sizeInArea, unsigned int clusterSize);
int bfs_mount(char *diskname, int debug);
int bfs_umount();
int disk_create(char *diskname, int nblocks);
```

4.3.2.1 File creation

So, what we (the users) will do in our program is to "travel" to some directory[2] (that will become the current directory, **cwd**) and call **bfs_create("FC")**. That will need to:

a) call **dir_ops.create(..., "FC", REG_FILE)**; *(if we were to create a dir, we would use **REG_DIR**)*
   a. the above function will scan the directory to see if there is a file/dir with the same name, else it will scan for a free entry
   b. call **ffs_ops.create(REG_FILE)**;
   c. If successful and metadata changes are synchronous, immediately save the pertinent information (bytemap, i-node and dentry changes) to disk;

b) The END.

From the above list, it should be obvious that we must implement all bytemap, i-node and directory operations first! Notice that the dentry structure size allows it to fit perfectly in a block, so you do not need – for now – to use the file's **read** and **write** operations for "small directories" (that fit in a cluster).

4.3.2.2 File open

So, no we (the users) call **bfs_ops.open("FC")**; that will need to:

a) use **dir_ops.readdir** to scan the directory for that filename, and returning the *dentry* or *dentry pointer*, when found;

b) Now, with the dentry we have the i-node. From there, we just call the FFS open function…

c) The END.

[2] Recall that **mount()** already sets the **cwd** to be the root directory's i-node for the disk

5. The END

For now… we will probably expand some of the previous sections to include and/or expand the existing descriptions, as well as adding new ones…

6. Piecewise delivery of the ZIPs and/or this Assignment

We will be delivering the following files in the ZIP:

```
disk_driver.{h, c}
ffs_super.{h, c}
ffs_bytemap.{h, c}
ffs_inode.{h, c}
ffs_file.{h, c}
bfs_errno.h
bfs_dir.{h, c}
bfs_lib.{h, c}
tstSB.c
tstLib*.c // several files may be delivered for your tests
Makefile
```

And, in a directory named **`Scripts`**, several test scripts where those ending in **`.in`** are the inputs for **`tstSB.c`** and those ending in **`.out`** are the expected outputs for correct implementations of FFS/BFS functions.

The colour RED is used for the files which contain /*** TODO ***/ entries. That's the files that you need to deliver in your ZIP!

Test programs How-To

The programs used to test BFS can be:

a) **tstSB**, a program that reads an input "specfile" and executes every action;
b) **tstCopy*.c**;
c) a mix of both;
d) a program that you, yourself, create (such as the one below).

The basic idea is that programs are created using the lib interface (see 4.3.2); an example is the one below, that creates a disk and formats it, asking you for some input…

```
int main(int argc, char *argv[]) {
  char diskname[8];
  int nblocks, cluster, fd, fd1, fd2, block_size;

  printf("Disk name (for create)? "); scanf("%s", diskname);
  printf("Disk size (for create)? "); scanf("%d", &nblocks);

  disk_create(diskname, nblocks);

  printf("I-node area size (in blocks)? "); scanf("%d", &nblocks);
  printf("Cluster size (in blocks, c=2^p)? "); scanf("%d", &cluster);
  bfs_format(diskname, nblocks, cluster);
}
```

Now, using **tstSB** you can display the on-disk information (if the answers were **disk0**, **1000**, **1**, **1**)

```
tstSB < spec0.in
```

you get

```
In-Memory Superblock:

  dirty            = no
  fsmagic          = 0xf0f03410
  nblocks          = 1000
  startInBmap      = 1
  sizeInBmap       = 1
  startInArea      = 2
  sizeInArea       = 1
  ninodes          = 32
  startDtBmap      = 3
  sizeDtBmap       = 2
  clusterSize      = 1
  startDtArea      = 5
  nclusters        = 995
  mounted          = no
In-Memory Superblock:
  ...             ...
  mounted          = yes

Printing the inode bytemap ----------
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Printing the data blocks bytemap ----------
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Name: . inode: 0
Name: .. inode: 0
Open a dir: Y 1 -1      ERROR
```

The “error” in the last line is not really an error (if your root directory has been created/updated correctly) but just the end of the entries in the directory 😊 Also, you can see that i-node 0 and data block 0 are already allocated (why? 😊)

The file **spec0.in** is

```
###
#This spec may be used to create and format a disk
#if you uncomment the D and F lines
#As is, it mounts a disk named disk0, then
#prints both bytemaps' entries and prints the
#root directory entries

#D disk0 1000

#Format args[1]= disk name, args[2]= sizeInArea,
#       args[3]= clusterSize

#F disk0 1 1

#Mount disk args[1]= disk name args[2]= debug if 1, not if 0
M disk0 1

#Print inodes bytemap
B 0
#Print data blocks bytemap
B 1

# Print vakid names in dir
Y 1

# Unmount the disk
U
```

You can create your own specfiles; the commands available (1 letter for each command) are:

```
D       disk: Disk create or open.
F       disk: Format a disk
M       disk: Mount
U       disk: Unmount

B       bytemap: print Bytemap table
g       bytemap: get free entry
s       bytemap: set entry to value

a       i-node: Allocate (just update=read-before-write an i-node)
d       i-node: Deallocate (just update=read-before-write a cleared i-node)
I       i-node: print Inode table

#       Comment (don’t use long comments, it will fail ☹)


---------- NEW commands, FFS
i       file: create a fIle (regular or dir)
u       file: delete/Unlink a file (regular or dir)
o       file: Open a file (regular or dir)
c       file: Close a file (regular or dir)
W       file: Write fd, <char>, I/O size (char populates write buffer)
R       file: Read  fd, <char>, I/O size (char populates compare buffer,
read reads to different buffer)
```

```
---------- NEW commands, DIR
#T      directory: create root dir (Do not use, format does it)
O       directory: Open file, by name
Y       directory: print directorY (all/valid entries)
e       directory: create dirEctory by name
f       directory: create File by name
C       directory: Cd to a subdirectory (DO NOT USE)
```